

UNIVERSITÉ
SORBONNE PARIS NORD

- [🏠 Portail](#) >
- [WSAIR2_2526](#) >
- [Groupes](#) >
- [Groupe 3](#) >
- [Wiki](#) >
- [Rendu-TP3](#) >
- [Page principale](#)

- [WSAIR2_2526](#)

Web Sémantique

WSAIR2_2526

Jérôme NOBECOURT

Groupe 3

[📁 Documents et liens](#) | [📁 Forums](#) | [📁 Wiki](#) | [💬 Discussion](#)

Groupe 3

Wiki : Rendu-TP3

- [Retour à la page](#)
- [Editer cette page](#)
- [Historique de cette page](#)

[📄 Page principale](#) | [🕒 Dernières modifications](#) | [📄 Liste des pages](#) | [📄 Liste des wikis](#) | [🔍 Rechercher](#)

Page principale

Compte-rendu TP3 — RDF, SPARQL et Corese

Web Sémantique · WSAIR2_2526 · Université Sorbonne Paris Nord

Table des matières

1. 1. Introduction et choix du sujet
1. 2. Modélisation de l'ontologie
1. 3. Le fichier RDF — glaces.rdf
1. 4. Visualisation du graphe RDF
1. 5. Installation et lancement de Corese

1. 6. Interrogation du graphe avec SPARQL

1. 7. Conclusion

1. Introduction et choix du sujet

L'objectif de ce TP était d'apprendre à utiliser des outils de recherche sémantique sur le web, en s'appuyant sur le moteur Corese. Le travail demandé consistait à rédiger un tutoriel personnel traduisant « en RDF » un sujet de notre choix, en réutilisant si possible un univers déjà étudié en TP1.

Le sujet choisi est l'univers de **La Compagnie des Glaces** (Georges-Jean Arnaud) : une terre recouverte de glace où des compagnies ferroviaires rivales font circuler des trains-villes entre des cités glaciaires. Cet univers se prête bien à une modélisation RDF car il met en jeu des entités clairement identifiables (personnages, villes, trains, compagnies) reliées par des relations simples (qui habite où, qui travaille pour qui, qui conduit quel train, quel train relie quelles villes).

Quatre fichiers ont été produits pour ce TP :

- `glaces.rdf` : l'ontologie et les données (RDF/XML) ;
- `q1_personnages.rq`, `q2_trains_par_compagnie.rq`, `q3_villes_froides.rq`, `q4_conducteurs.rq` : quatre requêtes SPARQL interrogeant ce graphe.

2. Modélisation de l'ontologie

Avant d'écrire le fichier RDF, il a fallu décider quelles classes et quelles propriétés représenter. Le choix s'est porté sur quatre classes et sept propriétés, suffisantes pour raconter les grandes lignes de l'univers sans le surcharger.

2.1 Les classes

- `glaces:Personnage` : un personnage de l'histoire (mécanicien, ingénieur, responsable...) ;
- `glaces:Ville` : une ville-terminus ou une ville-étape du réseau ;
- `glaces:Train` : un train-ville ou convoi appartenant à une compagnie ;
- `glaces:Compagnie` : une compagnie ferroviaire rivale.

Chaque classe est déclarée avec `rdfs:Class`, accompagnée d'un `rdfs:label` et d'un `rdfs:comment` en français, ce qui permet de documenter l'ontologie directement dans le fichier.

```
<rdfs:Class rdf:about="http://example.org/glaces#Personnage">
  <rdfs:label xml:lang="fr">Personnage</rdfs:label>
  <rdfs:comment xml:lang="fr">Un personnage de La Compagnie des Glaces.</rdfs:comment>
</rdfs:Class>
```

2.2 Les propriétés

- `glaces:nom` : nom littéral d'un personnage (domaine `Personnage`, portée `xsd:string`) ;
- `glaces:travaillePour` : relie un `Personnage` à la `Compagnie` qui l'emploie ;
- `glaces:habiteDans` : relie un `Personnage` à la `Ville` où il réside ;
- `glaces:conduire` : relie un `Personnage` au `Train` qu'il conduit ;
- `glaces:relie` : relie un `Train` aux `Villes` qu'il dessert ;
- `glaces:appartientA` : relie un `Train` à la `Compagnie` qui le possède ;
- `glaces:temperature` : température en °C d'une `Ville` (domaine `Ville`, portée `xsd:integer`).

Chaque propriété déclare un `rdfs:domain` et un `rdfs:range`, ce qui précise le type des ressources qu'elle relie. Par exemple, la propriété `conduire` ne relie qu'un `Personnage` à un `Train` :

```
<rdf:Property rdf:about="http://example.org/glaces#conduire">
  <rdfs:label xml:lang="fr">conduit</rdfs:label>
  <rdfs:domain rdf:resource="http://example.org/glaces#Personnage"/>
  <rdfs:range rdf:resource="http://example.org/glaces#Train"/>
</rdf:Property>
```

3. Le fichier RDF — `glaces.rdf`

Le fichier complet contient, après les classes et propriétés présentées ci-dessus, les individus suivants : deux compagnies, quatre villes, trois trains et quatre personnages.

3.1 Compagnies

```
<glaces:Compagnie rdf:about="http://example.org/glaces#CompagnieDesGlaces">
  <glaces:nom>La Compagnie des Glaces</glaces:nom>
  <rdfs:label xml:lang="fr">La Compagnie des Glaces</rdfs:label>
</glaces:Compagnie>

<glaces:Compagnie rdf:about="http://example.org/glaces#CompagnieDuNord">
  <glaces:nom>Compagnie du Nord</glaces:nom>
  <rdfs:label xml:lang="fr">Compagnie du Nord</rdfs:label>
</glaces:Compagnie>
```

3.2 Villes

Chaque ville porte une température, utilisée plus loin dans la requête 3.

```
<glaces:Ville rdf:about="http://example.org/glaces#NouvelleGeneve">
  <glaces:nom>Nouvelle-Genève</glaces:nom>
  <glaces:temperature rdf:datatype="http://www.w3.org/2001/XMLSchema#integer">-60</glaces:temperature>
  <rdfs:label xml:lang="fr">Nouvelle-Genève</rdfs:label>
  <rdfs:comment xml:lang="fr">Ville terminus principale de la Compagnie des Glaces.</rdfs:comment>
</glaces:Ville>

<glaces:Ville rdf:about="http://example.org/glaces#PortNord">
  <glaces:nom>Port-Nord</glaces:nom>
  <glaces:temperature rdf:datatype="http://www.w3.org/2001/XMLSchema#integer">-72</glaces:temperature>
  <rdfs:label xml:lang="fr">Port-Nord</rdfs:label>
  <rdfs:comment xml:lang="fr">Ville portuaire dans les zones arctiques.</rdfs:comment>
</glaces:Ville>

<glaces:Ville rdf:about="http://example.org/glaces#VilleCentrale">
  <glaces:nom>Ville-Centrale</glaces:nom>
  <glaces:temperature rdf:datatype="http://www.w3.org/2001/XMLSchema#integer">-50</glaces:temperature>
  <rdfs:label xml:lang="fr">Ville-Centrale</rdfs:label>
  <rdfs:comment xml:lang="fr">Nœud ferroviaire principal.</rdfs:comment>
</glaces:Ville>

<glaces:Ville rdf:about="http://example.org/glaces#LaTerminale">
  <glaces:nom>La Terminale</glaces:nom>
  <glaces:temperature rdf:datatype="http://www.w3.org/2001/XMLSchema#integer">-85</glaces:temperature>
  <rdfs:label xml:lang="fr">La Terminale</rdfs:label>
  <rdfs:comment xml:lang="fr">Ville la plus froide du réseau.</rdfs:comment>
</glaces:Ville>
```

3.3 Trains

Chaque train relie deux villes (propriété multivaluée relie) et appartient à une compagnie.

```
<glaces:Train rdf:about="http://example.org/glaces#ExpressPolaire">
  <glaces:nom>Express Polaire</glaces:nom>
  <glaces:relie rdf:resource="http://example.org/glaces#NouvelleGeneve"/>
  <glaces:relie rdf:resource="http://example.org/glaces#PortNord"/>
  <glaces:appartientA rdf:resource="http://example.org/glaces#CompagnieDesGlaces"/>
  <rdfs:label xml:lang="fr">Express Polaire</rdfs:label>
</glaces:Train>

<glaces:Train rdf:about="http://example.org/glaces#ConvoyeurCentral">
  <glaces:nom>Convoyeur Central</glaces:nom>
  <glaces:relie rdf:resource="http://example.org/glaces#VilleCentrale"/>
  <glaces:relie rdf:resource="http://example.org/glaces#LaTerminale"/>
  <glaces:appartientA rdf:resource="http://example.org/glaces#CompagnieDuNord"/>
  <rdfs:label xml:lang="fr">Convoyeur Central</rdfs:label>
</glaces:Train>

<glaces:Train rdf:about="http://example.org/glaces#GlacierExpress">
  <glaces:nom>Glacier Express</glaces:nom>
  <glaces:relie rdf:resource="http://example.org/glaces#NouvelleGeneve"/>
  <glaces:relie rdf:resource="http://example.org/glaces#VilleCentrale"/>
  <glaces:appartientA rdf:resource="http://example.org/glaces#CompagnieDesGlaces"/>
```

```
<rdfs:label xml:lang="fr">Glacier Express</rdfs:label>
</glaces:Train>
```

3.4 Personnages

Trois personnages conduisent un train ; le quatrième (Irina Voss) n'en conduit pas, ce qui permet de tester le caractère facultatif de la propriété conduire dans les requêtes.

```
<glaces:Personnage rdf:about="http://example.org/glaces#LucienBruk">
  <glaces:nom>Lucien Bruk</glaces:nom>
  <glaces:habiteDans rdf:resource="http://example.org/glaces#NouvelleGeneve"/>
  <glaces:travaillePour rdf:resource="http://example.org/glaces#CompagnieDesGlaces"/>
  <glaces:conduire rdf:resource="http://example.org/glaces#ExpressPolaire"/>
  <rdfs:label xml:lang="fr">Lucien Bruk</rdfs:label>
  <rdfs:comment xml:lang="fr">Mécanicien principal de l'Express Polaire.</rdfs:comment>
</glaces:Personnage>

<glaces:Personnage rdf:about="http://example.org/glaces#ElsaKorvac">
  <glaces:nom>Elsa Korvac</glaces:nom>
  <glaces:habiteDans rdf:resource="http://example.org/glaces#PortNord"/>
  <glaces:travaillePour rdf:resource="http://example.org/glaces#CompagnieDuNord"/>
  <glaces:conduire rdf:resource="http://example.org/glaces#ConvoyeurCentral"/>
  <rdfs:label xml:lang="fr">Elsa Korvac</rdfs:label>
  <rdfs:comment xml:lang="fr">Ingénieure en chef de la Compagnie du Nord.</rdfs:comment>
</glaces:Personnage>

<glaces:Personnage rdf:about="http://example.org/glaces#MarcTaiga">
  <glaces:nom>Marc Taiga</glaces:nom>
  <glaces:habiteDans rdf:resource="http://example.org/glaces#VilleCentrale"/>
  <glaces:travaillePour rdf:resource="http://example.org/glaces#CompagnieDesGlaces"/>
  <glaces:conduire rdf:resource="http://example.org/glaces#GlacierExpress"/>
  <rdfs:label xml:lang="fr">Marc Taiga</rdfs:label>
  <rdfs:comment xml:lang="fr">Conducteur expérimenté du Glacier Express.</rdfs:comment>
</glaces:Personnage>

<glaces:Personnage rdf:about="http://example.org/glaces#IrinaVoss">
  <glaces:nom>Irina Voss</glaces:nom>
  <glaces:habiteDans rdf:resource="http://example.org/glaces#LaTerminale"/>
  <glaces:travaillePour rdf:resource="http://example.org/glaces#CompagnieDuNord"/>
  <rdfs:label xml:lang="fr">Irina Voss</rdfs:label>
  <rdfs:comment xml:lang="fr">Responsable de la maintenance à La Terminale.</rdfs:comment>
</glaces:Personnage>
```

Le fichier a été vérifié à l'aide du validateur RDF du W3C (<http://www.w3.org/RDF/Validator/>), qui confirme la conformité syntaxique du document.

4. Visualisation du graphe RDF

Pour obtenir une vue d'ensemble du graphe, les triplets ont été traduits en un graphe orienté au format DOT (Graphviz), chaque classe étant représentée par une forme et une couleur différentes : ellipse bleue pour les personnages, rectangle vert arrondi pour les villes, losange jaune pour les trains, rectangle rouge pour les compagnies. Les arêtes portent le nom de la propriété RDF correspondante.

```
digraph GlacesRDF {
  rankdir=LR;
  node fontname="Helvetica", fontsize=11, style=filled;
  edge fontname="Helvetica", fontsize=9, color="#555555";

  // Personnages
  node shape=ellipse, fillcolor="#cfe8ff", color="#1a3d6d";
  LucienBruk label="Lucien Bruk";
  ElsaKorvac label="Elsa Korvac";
  MarcTaiga label="Marc Taiga";
  IrinaVoss label="Irina Voss";

  // Villes
  node shape=box, style="filled,rounded", fillcolor="#d6f5d6", color="#2e7d32";
  NouvelleGeneve label="Nouvelle-Genève\n-60°C";
  PortNord label="Port-Nord\n-72°C";
```

```

VilleCentrale label="Ville-Centrale\n-50°C";
LaTerminale label="La Terminale\n-85°C";

// Trains
node shape=diamond, fillcolor="#fff2cc", color="#b8860b";
ExpressPolaire label="Express Polaire";
ConvoyeurCentral label="Convoyeur Central";
GlacierExpress label="Glacier Express";

// Compagnies
node shape=box, style="filled,bold", fillcolor="#f8d7d7", color="#a33a3a";
CompagnieDesGlaces label="La Compagnie\ndes Glaces";
CompagnieDuNord label="Compagnie\ndu Nord";

LucienBruk -> NouvelleGeneve label="habiteDans";
LucienBruk -> CompagnieDesGlaces label="travaillePour";
LucienBruk -> ExpressPolaire label="conduit";
ElsaKorvac -> PortNord label="habiteDans";
ElsaKorvac -> CompagnieDuNord label="travaillePour";
ElsaKorvac -> ConvoyeurCentral label="conduit";
MarcTaiga -> VilleCentrale label="habiteDans";
MarcTaiga -> CompagnieDesGlaces label="travaillePour";
MarcTaiga -> GlacierExpress label="conduit";
IrinaVoss -> LaTerminale label="habiteDans";
IrinaVoss -> CompagnieDuNord label="travaillePour";

ExpressPolaire -> NouvelleGeneve label="relie";
ExpressPolaire -> PortNord label="relie";
ExpressPolaire -> CompagnieDesGlaces label="appartientA";
ConvoyeurCentral -> VilleCentrale label="relie";
ConvoyeurCentral -> LaTerminale label="relie";
ConvoyeurCentral -> CompagnieDuNord label="appartientA";
GlacierExpress -> NouvelleGeneve label="relie";
GlacierExpress -> VilleCentrale label="relie";
GlacierExpress -> CompagnieDesGlaces label="appartientA";
}

```

Ce code DOT a été déposé dans l'outil en ligne <https://dreampuf.github.io/GraphvizOnline> pour obtenir le rendu SVG du graphe (fichier joint `glaces_graph.svg`). On y voit clairement les quatre personnages au centre-gauche, reliés aux villes, aux trains qu'ils conduisent et aux compagnies qui les emploient, ainsi que les trains reliant chacun deux villes et appartenant à une compagnie.

[Image: [glaces_graph.svg](#)]

5. Installation et lancement de Corese

Corese nécessite au minimum le JRE 8. La version utilisée est la version standalone de Corese (Core V3), distribuée sous forme de fichier `.jar`.

Le jar se lance depuis un terminal, dans le dossier où se trouve le fichier :

```
java -jar corese-3.2.3c.jar
```

Une interface graphique s'ouvre, permettant de :

- charger le graphe RDF (File > Load > RDF/XML), ici `glaces.rdf` ;
- charger une requête SPARQL (File > Load > SPARQL), ou la saisir directement dans l'éditeur de requêtes ;
- exécuter la requête et visualiser le tableau de résultats dans l'onglet de résultats.

Le chargement de `glaces.rdf` ne déclenche aucune erreur, ce qui confirme une nouvelle fois la validité syntaxique du fichier. Une fois le graphe chargé, chacune des quatre requêtes ci-dessous a été collée dans l'éditeur SPARQL de Corese puis exécutée.

6. Interrogation du graphe avec SPARQL

6.0 Requête pour visualiser le graphe RDF complet dans Corese

Pour afficher l'ensemble du graphe RDF dans Corese, on utilise une requête SPARQL de type CONSTRUCT. Cette requête permet de reconstruire tous les triplets du graphe afin de générer une visualisation globale.

PREFIX glaces: <http://example.org/glaces#>

```
CONSTRUCT {
  ?s ?p ?o .
}
WHERE {
  ?s ?p ?o .
}
```

Cette requête sélectionne tous les triplets RDF du modèle :

- ?s correspond au sujet (ressource RDF)
- ?p correspond à la propriété (prédicat)
- ?o correspond à l'objet (valeur ou ressource)

Elle permet donc de reconstruire l'intégralité du graphe RDF sans filtrage ni restriction.

Dans Corese, l'exécution de cette requête permet d'afficher le graphe complet dans la vue graphique ou de l'exporter (par exemple en SVG).

6.1 Requête 1 — Personnages, villes et compagnies

Cette première requête liste tous les personnages avec leur ville de résidence et leur compagnie employeur.

PREFIX glaces: <http://example.org/glaces#>
 PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

```
SELECT ?nom ?ville ?compagnie
WHERE {
  ?p a                glaces:Personnage ;
     glaces:nom       ?nom ;
     glaces:habiteDans ?v ;
     glaces:travaillePour ?c .
  ?v glaces:nom       ?ville .
  ?c glaces:nom       ?compagnie .
}
ORDER BY ?nom
```

Le motif de base sélectionne toute ressource typée `glaces:Personnage` possédant un nom, une ville (`habiteDans`) et une compagnie (`travaillePour`). Les deux dernières lignes du `WHERE` remontent ensuite le nom littéral de la ville et de la compagnie à partir de leurs URI respectives. Le tri final se fait sur le nom du personnage.

Comme tous les personnages possèdent une ville et une compagnie, les quatre personnages du graphe apparaissent dans le résultat — y compris Irina Voss, qui ne conduit pourtant aucun train.

Résultat de la requête 1

nom	ville	compagnie
Elsa Korvac	Port-Nord	Compagnie du Nord
Irina Voss	La Terminale	Compagnie du Nord
Lucien Bruk	Nouvelle-Genève	La Compagnie des Glaces
Marc Taiga	Ville-Centrale	La Compagnie des Glaces

6.2 Requête 2 — Trains par compagnie

Cette requête liste, pour chaque compagnie, les trains qui lui appartiennent ainsi que les villes que chaque train relie.

PREFIX glaces: <http://example.org/glaces#>

```
SELECT ?compagnie ?train ?ville
WHERE {
  ?t a                glaces:Train ;
     glaces:nom       ?train ;
     glaces:appartientA ?c ;
     glaces:relie     ?v .
  ?c glaces:nom       ?compagnie .
  ?v glaces:nom       ?ville .
}
ORDER BY ?compagnie ?train
```

Comme la propriété `glaces:relie` est multivaluée (chaque train relie deux villes), le motif `?t glaciers:relie ?v` produit une ligne de résultat par ville desservie : chaque train apparaît donc deux fois dans le tableau, une fois par ville reliée. Le tri se fait d'abord sur le nom de la compagnie, puis sur le nom du train.

Résultat de la requête 2

compagnie	train	ville
Compagnie du Nord	Convoyeur Central	Ville-Centrale
Compagnie du Nord	Convoyeur Central	La Terminale
La Compagnie des Glaces Express	Polaire	Nouvelle-Genève
La Compagnie des Glaces Express	Polaire	Port-Nord
La Compagnie des Glaces Glacier Express		Nouvelle-Genève
La Compagnie des Glaces Glacier Express		Ville-Centrale

6.3 Requête 3 — Villes les plus froides

Cette requête classe les villes par température croissante, c'est-à-dire de la plus froide à la moins froide.

PREFIX `glaces:` <<http://example.org/glaces#>>

PREFIX `xsd:` <<http://www.w3.org/2001/XMLSchema#>>

```
SELECT ?ville ?temperature
WHERE {
  ?v a                glaciers:Ville ;
      glaciers:nom     ?ville ;
      glaciers:temperature ?temperature .
}
ORDER BY ASC(?temperature)
```

Comme la propriété `glaces:temperature` est typée `xsd:integer`, le tri `ASC(?temperature)` s'effectue numériquement (et non alphabétiquement) : c'est ce typage qui garantit que -85 est bien considéré comme plus petit que -50.

Résultat de la requête 3

ville	temperature
La Terminale	-85
Port-Nord	-72
Nouvelle-Genève	-60
Ville-Centrale	-50

On retrouve bien La Terminale en tête, conformément à sa description dans le fichier RDF (« Ville la plus froide du réseau »).

6.4 Requête 4 — Conducteurs et villes desservies

Cette dernière requête relie chaque conducteur au train qu'il conduit, puis aux villes que ce train dessert.

PREFIX `glaces:` <<http://example.org/glaces#>>

```
SELECT ?conducteur ?train ?villeDesservie
WHERE {
  ?p a                glaciers:Personnage ;
      glaciers:nom     ?conducteur ;
      glaciers:conduire ?t .
  ?t glaciers:nom     ?train ;
      glaciers:relie   ?v .
  ?v glaciers:nom     ?villeDesservie .
}
ORDER BY ?conducteur
```

La propriété `glaces:conduire` étant facultative (Irina Voss ne la possède pas), seuls les trois personnages qui conduisent effectivement un train apparaissent dans le résultat. Comme dans la requête 2, chaque train reliant deux villes, chaque conducteur apparaît deux fois.

Résultat de la requête 4

conducteur	train	villeDesservie
Elsa Korvac	Convoyeur Central	Ville-Centrale
Elsa Korvac	Convoyeur Central	La Terminale
Lucien Bruk	Express Polaire	Nouvelle-Genève
Lucien Bruk	Express Polaire	Port-Nord
Marc Taiga	Glacier Express	Nouvelle-Genève

Marc Taiga Glacier Express Ville-Centrale

Irina Voss n'apparaît dans aucune ligne de ce résultat, ce qui illustre la différence entre une jointure simple (requête 1, où `habiteDans` et `travaillePour` sont obligatoires pour tous) et une jointure sur une propriété facultative (requête 4, où l'absence de `conduire` exclut silencieusement le personnage du résultat).

7. Conclusion

Ce TP a permis de mettre en pratique l'ensemble de la chaîne du Web sémantique sur un exemple concret. La modélisation de l'univers de **La Compagnie des Glaces** a d'abord nécessité de choisir un petit nombre de classes (Personnage, Ville, Train, Compagnie) et de propriétés (nom, `habiteDans`, `travaillePour`, `conduire`, `relie`, `appartientA`, `temperature`) suffisant pour représenter les relations essentielles de l'univers sans complexité inutile.

L'écriture du fichier `glaces.rdf` en RDF/XML a ensuite permis de traduire ce modèle en un graphe de triplets, validé syntaxiquement à l'aide du validateur du W3C puis visualisé sous forme de graphe orienté grâce à Graphviz.

Enfin, l'utilisation de Corese pour charger ce graphe et y exécuter quatre requêtes SPARQL a permis d'illustrer plusieurs constructions classiques du langage : jointures multiples entre motifs de triplets (requête 1), gestion des propriétés multivaluées produisant plusieurs lignes de résultat (requêtes 2 et 4), tri numérique grâce au typage XSD (requête 3), et effet d'une propriété facultative sur l'ensemble de résultat (requête 4, où Irina Voss disparaît faute de propriété `conduire`).

Ce TP montre ainsi comment un univers narratif, même simple, peut être structuré en ontologie RDF/RDFS, puis interrogé efficacement à l'aide de SPARQL et d'un moteur comme Corese.

Gestionnaire(s) de WSAIR2_2526 : Jérôme NOBECOURT

Administrateur de Portail : .Webmestre

Utilise la plateforme Claroline © 2001 - 2013

modifiée depuis 2013 à aujourd'hui pour ce portail